

MACHINE LEARNING

HOMEWORK – 1

Ronitt Mehra

rm4084

Hunter Wilder

hdw2112

Question-1

A. Assuming the two images be X and Y.

Let,

X be the image in which all pixels are at the minimum intensity (0)

Y be the image in which all pixels are at the maximum intensity (255).

Since the MNIST dataset consists of images of dimensions 28 X 28, it is obvious that each image in the dataset has $28 \times 28 = 28^2 = 784$ pixels

Now, each pixel in image X has intensity of 0, and each pixel in image Y has intensity 255. So, in totality, image X has 784 pixels of intensity 0, and image Y has 784 pixels of intensity 255.

Let, x_i – pixel intensity of the i^{th} pixel in image X

y_i – pixel intensity of the i^{th} pixel in image Y

So, x is the 784-dimensional vector of the pixel intensity of each of the pixels in image X.

y is the 784-dimensional vector of the pixel intensity of each of the pixels in image Y.

$x = [0, 0, 0, \dots, 0]$

$y = [255, 255, 255, \dots, 255]$

For computing the Euclidean distance, we need to first compute the sum of the squared deviation of the pixel intensity of each pixel in image X (x_i) to its corresponding pixel in image Y (y_i). Afterwards, we take a square-root of the resultant sum to finally obtain the Euclidean distance between these two images.

Mathematically,

Euclidean distance =

$$\sqrt{\sum_{i=1}^{i=784} (x_i - y_i)^2}$$

$$\begin{aligned}
&= \sqrt{\sum_{i=1}^{784} (0 - 255)^2} \\
&= \sqrt{784 * (255)^2} \\
&= \sqrt{(28)^2 * (255)^2} \\
&= \sqrt{(28 * 255)^2} \\
&= 28 * 255 \\
&= 7140
\end{aligned}$$

B.

It's given that $N(x; S_{MNIST})$ is basically the Nearest Neighbour for an image 'x' in the test set. We know that the Euclidean distance of an image from any image in the training set is the minimum for its nearest neighbour.

The Euclidean distance between an image 'x' in test set and its nearest neighbour in S_{MNIST} , is given by:

$$||x - NN(x; S_{MNIST})||$$

Therefore, $||x - NN(x; S_{MNIST})||^2$ is simply the square of the Euclidean distance between 'x' and its nearest neighbour.

Python code:

```
import math
import numpy as np
NN_distances = []

for i in range(len(mnist['testdata'])):
    NN_Euclidean_Distance = 200000
    for j in range(len(mnist['data'])):
        temp = mnist['testdata'][i] - mnist['data'][j]
        euc_dist = np.sqrt(np.dot(temp.T, temp))
        if(euc_dist < NN_Euclidean_Distance):
            NN_Euclidean_Distance = euc_dist

    NN_distances.append(NN_Euclidean_Distance)

sum = 0
for i in NN_distances:
    sum = sum + pow(i,2)
```

```
average = sum/len(mnist['testdata'])
print("Average: ", average)

square_root = NN_distances
square_root = np.array(square_root)
print("Square root: ", square_root)
```

Output:

```
Average: 1254922.3729
Square root: [ 676.58406721 1162.9316403  321.66286699 ...  985.55415884 1313.7682444
1014.89457581]
```

Explanation:

First, the Euclidean distance of each image in the test set is computed with each image in the training dataset. The minimum of the Euclidean distance has been computed (this is the distance of the image to its nearest neighbour), and stored in a list named, 'NN_distances'.

For computing $\|x - NN(x; S_{MNIST})\|^2$, each element in the above created list is squared and added. The result is then divided by the total number of samples in the testing set, to finally compute the average.

Furthermore, the square root would basically give us the Euclidean distance of each of the images in the testing set with its nearest neighbour in the training set.

C.

Upon augmenting the images with 280 randomly generated pixels, the dimensions of the image change from 28 X 28 to 28 X 38. Upon flattening the image into a one-dimensional vector, every image is represented by a (1064,1) vector.

Below is the python code for augmenting the images, and then applying Nearest Neighbour classification, and computing the test error rate.

Python Code:

```
import random
mnist['data'] = mnist['data'].astype(int)
# creating a list that contains augmented images corresponding to the
training data
train = []

# Augmenting each of the training images in mnist dataset with 280
noisy pixels
for i in range(len(mnist['data'])):
    pixels = []
    for j in range(280):
```

```

        #intensity of random pixel varies between 0 and 255
        pixels.append(random.randint(0,255))
    train.append(np.hstack((mnist['data'][i], pixels)))

train = np.array(train)
for i in range(len(train)):
    train[i] = np.array(train[i])

mnist['testdata'] = mnist['testdata'].astype(int)

# creating a list that contains augmented images corresponding to the
testing data
test = []

for i in range(len(mnist['testdata'])):
    pixels = []
    for j in range(280):
        pixels.append(random.randint(0,255))
    test.append(np.hstack((mnist['testdata'][i], pixels)))

test = np.array(test)
for i in range(len(test)):
    test[i] = np.array(test[i])

from sklearn.neighbors import KNeighborsClassifier
# Deploying the Nearest Neighbor classifier
model = KNeighborsClassifier(1)
# Training the Nearest Neighbor classifier over training data
model.fit(train, mnist['labels'])
# calculating accuracy of predictions using the testing data
accuracy = model.score(test, mnist['testlabels'])
print("Test Error Rate: ", round((1-accuracy)*100, 2), "%")

```

Output:

```
Test Error Rate:  3.9 %
```

Therefore, the Test Error Rate of Nearest Neighbour with these augmented images is 3.9%.

D.

For the k-NN algorithm where $k=n$ (number of training data points) the classification will just depend on the most-frequently occurring number in the whole training set.

Python Code:

```
import pickle as p
import numpy as np
mnist = p.load(open('mnist.pkl', 'rb'))

#print(int(mnist['labels'][1]))
#import matplotlib.pyplot as plt
#plt.imshow(mnist['data'][1].reshape(28,28), cmap='gray_r') # show
#first image
#plt.show()
#print(type(list(mnist['labels'])))

nTotals = [0,0,0,0,0,0,0,0,0,0] #list of # of each number that shows up
in sat
mlist=list(mnist['labels']) #casting to list
for i in mlist: #iterate through each item in the list
    nTotals[i]+=1 #add 1 to the index of 'nTotal' that corresponds to
the number
print(nTotals) #return list of the counts of each number
print('The result of k-NN where k = n
is:',nTotals.index(np.max((nTotals))))
#training error rate = ((Number of training samples incorrectly
prredicted)*100/(total number of training samples))
print('Training error rate for k = n: ', ((len(mnist['data']) -
nTotals[nTotals.index(np.max((nTotals)))])/len(mnist['data']))*100,
"%")
```

Output:

```
[5923, 6742, 5958, 6131, 5842, 5421, 5918, 6265, 5851, 5949]
The result of k-NN where k = n is: 1
Training error rate for k = n: 88.76333333333334 %
```

In the Python code, we have first computed the occurrences of each of the digit in the training set. It is observed that 1 is the most occurring digit, with an occurrence count of 6742.

Thus, using the property that the most-frequently occurring (mode) digit is 1, we were able to compute the training error rate ≈ 88.7634

Question – 2

a.

To answer this question, we need to ask how to verify whether there are some two items belonging to the output space that when compared to an input will return different results depending on the metric. To do this I'll consider 2 almost identical items (X_1, X_2):

$$D_a(x_1, z_1) = ||x_1 - z_1|| = (\epsilon) + (\chi)$$

$$D_a(x_2, z_2) = ||x_2 - z_2|| = (2\epsilon) + (\chi - \delta)$$

Where,

$$\delta < \epsilon \ll \chi$$

So, with these 2 items the dots represent the fact that these objects are almost identical and only differ in the 2 places noted and in the case of the 4th order Minkowski distance this goes like:

$$D_b(x_1, z_1) = ||x_1 - z_1||_4 = ((\epsilon)^4 + (\chi)^4)^{\frac{1}{4}}$$

$$D_b(x_2, z_2) = ||x_2 - z_2||_4 = ((2\epsilon)^4 + (\chi - \delta)^4)^{\frac{1}{4}}$$

Now in the case of D_a it is clear to see that $D_a(x_1, z_1) < D_a(x_2, z_2)$, but in the case of D_b I will now make the case that the opposite is true:

$$\begin{aligned} D_b(x_2, z_2) &= ||x_2 - z_2||_4 = ((2\epsilon)^4 + (\chi - \delta)^4)^{\frac{1}{4}} \\ &= (\chi - \delta) \left(\left(\frac{2\epsilon}{\chi - \delta} \right)^4 + 1 \right)^{\frac{1}{4}} \\ &\approx (\chi - \delta) \left(\frac{1}{4} \left(\frac{2\epsilon}{\chi - \delta} \right)^4 + 1 \right) \\ &\approx \left(\left(\frac{4\epsilon^4}{(\chi - \delta)^3} \right) + (\chi - \delta) \right) \\ &\approx \left(\left(\frac{4\epsilon^4}{\chi^3(1 - \delta/3\chi)} \right) + (\chi - \delta) \right) \end{aligned}$$

And we can make a similar approximation with $D_a(x_1, z_1)$:

$$\begin{aligned} D_b(x_1, z_1) &= ||x_1 - z_1||_4 = ((\epsilon)^4 + (\chi)^4)^{\frac{1}{4}} \\ &\approx \chi \left(1 + \frac{\epsilon^4}{4\chi^4} \right) \\ &\approx \left(\chi + \frac{\epsilon^4}{4\chi^3} \right) \end{aligned}$$

And now after making this approximation for χ being much greater than δ and ϵ we will assume the opposite of the relation between $D_a(x_1, z_1)$ and $D_a(x_2, z_2)$ see that we do not contradicted ourselves and there for the different distance formulas have yielded different classifications:

$$D_a(x_1, z_1) > D_a(x_2, z_2)$$

$$\left(\chi + \frac{\epsilon^4}{4\chi^3}\right) > \left(\left(\frac{4\epsilon^4}{\chi^3(1 - \delta/3\chi)}\right) + (\chi - \delta)\right)$$

$$\left(\frac{1}{4\chi^3} - \frac{4}{\chi^3(1 - \delta/3\chi)}\right)\epsilon^4 + \chi > (\chi - \delta)$$

And here we see that we have no contradiction because the first term on the left is either a positive or negative quantity multiply by a very small number relation to χ and so it's ultimately just a very small contribution on the left that we might as well say is 0 compared to χ .

$$\chi > (\chi - \delta)$$

$\therefore$

$$D_a(x_1, z_1) > D_a(x_2, z_2)$$

And here I think it's clear that obviously χ is great than χ minus some quantity and so we've found that it is true that the classifier that is use the 4th order Minkowski distance will classify things different from the 1st order distance.

b.

Squaring the Manhattan distance will change the classification because if we analyze the same marginal case:

the classification in the normal Manhattan distance case is the same part a: $D_a(x_1, z_1) < D_a(x_2, z_2)$

But for the squared Manhattan distance it runs into a similar problem:

$$(2\epsilon + (\chi - \delta))^2 = 4\epsilon^2 + 2\epsilon\chi - 2\epsilon\delta + \chi^2 - 2\chi\delta + \delta^2$$

$$\approx \chi^2 + 2(\epsilon - \delta)\chi$$

$$\approx \chi + 2\epsilon\chi$$

and so, it's clear enough to see that:

$$\chi + 2\epsilon\chi > \chi^2 + 2(\epsilon - \delta)\chi$$

and so:

$$||x_1 - z_1|| < ||x_2 - z_2||$$

and,

$$||x_1 - z_1||^2 > ||x_2 - z_2||^2$$

and so, the same conclusion can be drawn that the classifiers using these different metrics will not reach the same classifications for all cases.

c.

if:

$$||x - z||_1 = 1$$

then:

$$|x_i - z_i| < 1$$

and so, we then know that.

$$|x_i - z_i|^3 \ll |x_i - z_i|$$

and therefore, we can conclude that $(\sum |x_i - z_i|^3)^{\frac{1}{3}}$ will most likely be smaller than $(\sum |x_i - z_i|)^{\frac{1}{3}} = 1$ which means that $||x - z||_3$ is at most 1 but probably less.

Question – 3

a.

To find the chance that $Y_1 = 1$ this will have to be the chance that one has covid and answers truthfully plus the probability that one doesn't have covid and gives the opposite response. And so the chance the one flips a coin 4 times and get either 2 or 3 heads is:

$$\frac{4!}{2!2!} = \frac{3}{8}, \text{ for 2 heads}$$

$$\frac{4!}{3!1!} = \frac{2}{8}, \text{ for 3 heads}$$

$$\frac{3}{8} + \frac{2}{8} = \frac{5}{8}, \text{ for 3 or 2 heads}$$

so, the probability that one has covid and tells the truth is:

$$\frac{5}{8}\theta$$

And the probability one doesn't have covid and lies:

$$\frac{3}{8}(1 - \theta)$$

so, the probability in general that we get $Y_1 = 1$:

$$\frac{5}{8}\theta + \frac{3}{8}(1 - \theta) = \frac{2\theta + 3}{8}$$

b.

the expression for the log likelihood of some parameter θ given a set of output data is given by:

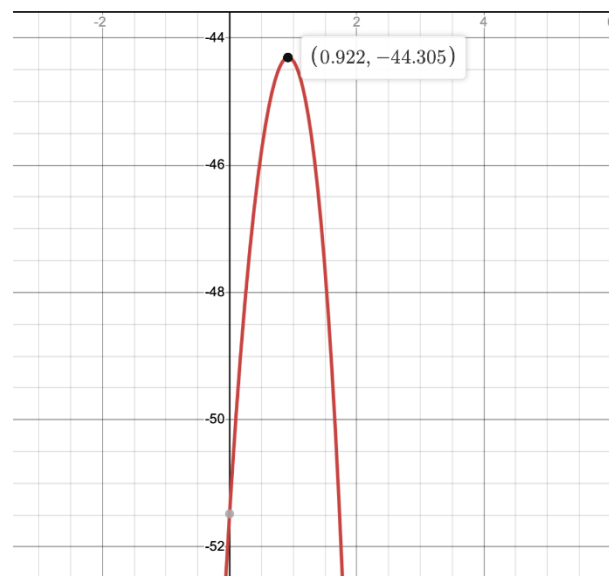
$$\sum_i \log(\Pr(Y = y_i|\theta))$$

c.

If we feed in the numbers, we are given here then we get:

$$\sum_i \log(\Pr(Y = y_i|\theta)) = \sum_{i=1}^{40} \log\left(\frac{2\theta + 3}{8}\right) + \sum_{i=41}^{100} \log\left(\frac{5 - 2\theta}{8}\right)$$

Plotting this sum with respect to θ gives:



This is the plot produce by the log-likelihood

And so, we see there is a peak θ value at 0.922.

Question – 4

To proceed with this question, we have created a plot on Python using the classifier provided in the question. The parameters provided in the question have been used for the purpose of plotting the distribution of x, corresponding to the different labels, i.e., 1, 2, 3, for Setosa, Versicolor, and Virginica respectively.

Python Code:

```
import numpy as np
import pandas as pd
from sklearn import datasets
#loading iris dataset
iris = datasets.load_iris()
import math
# matplotlib used for creating the gaussian plots
import matplotlib.pyplot as plt
#print(iris['data'])
min = 1000000
for i in range(len(iris['data'])):
    if(iris['data'][i][0] < min):
        min = iris['data'][i][0]

# min - the minimum sepal length in iris dataset
print("Minimum Sepal Length in Iris dataset: ", min)

max = 0
for i in range(len(iris['data'])):
    if(iris['data'][i][0] > max):
        max = iris['data'][i][0]

#max - maximum sepal length in iris dataset
print("Maximum Sepal Length in the Iris dataset: ", max)

min = min.astype('float')
max = max.astype('float')
# set a range of 'x' for the generative modelling
# the sepal length is varied between the minimum and the maximum
x = [min, max]
ycor1 = []
xcor = []
y1 = []
while (min<=max):
    xcor.append(min)
    # creating the classifier using parameters obtained through MLE,
    provided in the question
```

```

    f = (1/3)*(1/math.sqrt(2*math.pi*0.68*0.68))*math.exp(-1*pow((min -
6.61),2)/(2*0.68*0.68))
    ycor1.append(f)
    min+=0.01
    y1.append((min, f))

#plt.plot(xcor, ycor1, label = 'density vs sepal length plot', color =
'blue')
min = x[0]
max = x[1]
xcor = []
ycor2 = []
y2 = []
while(min<=max):
    xcor.append(min)
    # creating the classifier using parameters obtained through MLE,
provided in the question
    f = (1/3)*(1/math.sqrt(2*math.pi*0.47*0.47))*math.exp(-1*pow((min -
5.93),2)/(2*0.47*0.47))
    ycor2.append(f)
    min+=0.01
    y2.append((min, f))

plt.plot(xcor, ycor1, label = 'density vs sepal length plot', color =
'blue')
plt.plot(xcor, ycor2, label = 'density vs sepal length plot', color =
'red')
plt.xlabel("Sepal Length")
plt.ylabel("Density")
plt.title("Density vs Sepal Length plot for the different species of
iris")
intersection = 0
#finding the point of intersection between f = 2 and f = 3
min = 100000000

for x in range(len(y1)):
    if(abs(y1[x][1] - y2[x][1]) < min and y1[x][0] >= 5 and y1[x][0] <=
7):
        intersection = y1[x][0]
        min = abs(y1[x][1] - y2[x][1])

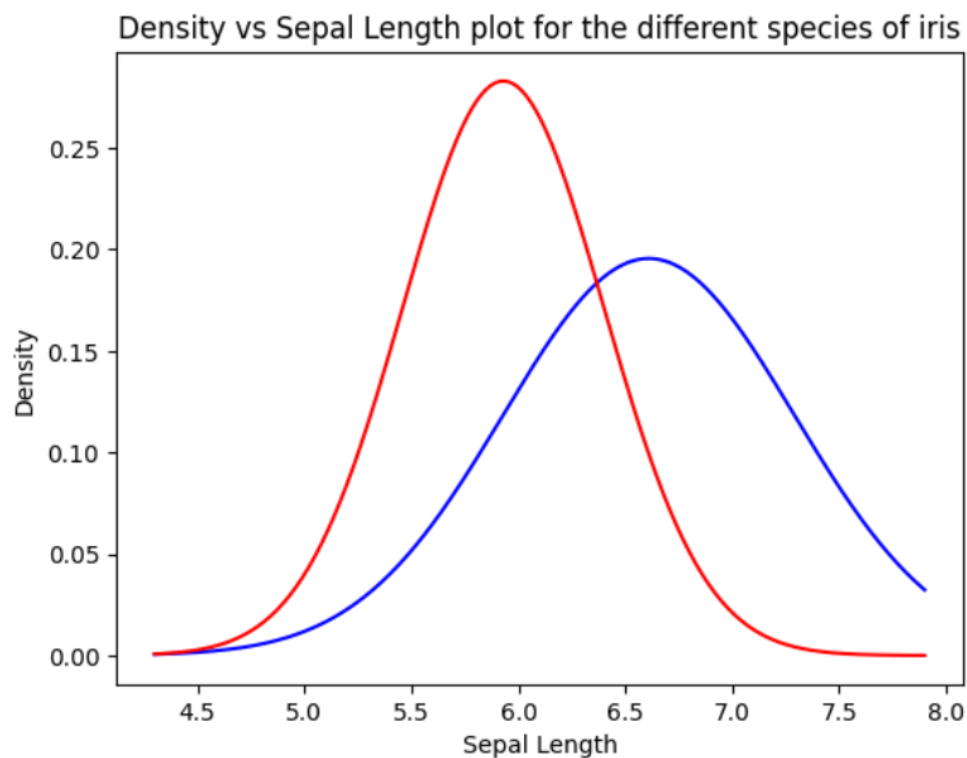
print("Intersection point between f = 2 and f = 3: ", intersection)
print("Therefore, a = ", intersection)
print("b = ", max)

```

Output:

```
Minimum Sepal Length in Iris dataset:  4.3  
Maximum Sepal Length in the Iris dataset:  7.9
```

```
Intersection point between f = 2 and f = 3:  6.3799999999999555  
Therefore, a =  6.3799999999999555  
b =  7.9
```



In our plot above, the red distribution is corresponding to $\hat{f}(x) = 2$, and the blue distribution is corresponding to $\hat{f}(x) = 3$

Therefore, from the plot, for $\hat{f}(x) = 2$ and $\hat{f}(x) = 3$, there is an intersection point at

$$x \approx 6.38$$

Thus,

$$a \approx 6.38$$

$$b = 7.9$$

Hence, set of x's for which $\hat{f}(x) = 3$ is $[6.38, 7.9]$

Question – 5

Qualitatively, we would expect that the classifier will now classify less setosa and more of the other two as the loss function the expected loss function will take the form of something like:

$$E[\hat{f}(\hat{y}, y)] = Pr(\hat{y} = 2|y = 1) + Pr(\hat{y} = 3|y = 1) + 2Pr(\hat{y} = 1|y = 2) + 2Pr(\hat{y} = 1|y = 3)$$

And this will be minimized at the minimums of each respective probability which then tells us that the region of setosa should essentially take a bit shorter of a range than it does by default because the classifier will want to minimize the risk which will in turn minimize the amount of times that setosa is misclassified as one of the others and therefore will have an effect of strinking the width of the gaussian of setosa as to minimize the overlap of its probability with that of any of the others.

Question – 6

- A.** The collection of articles published last year in the newspaper, labelled with the number of views each article received, is decent training data for predicting the number of views an article will get. In this scenario, if we assume that the articles published last year are very similar to the articles published this year, the model created would provide a high accuracy, because of many similar words in the articles published last year, and those published this year. However, if the articles published last year are not at all like the ones published this year, the model would provide a poor accuracy, because the model would essentially not be appropriately trained for the testing data. It would be wiser to incorporate other features as well, such as genre of the articles, name of journalist, etc, into the classification model, as these features would be better representatives of the population and would give rise to a more accurate model.
- B.** The college applications of students from 2018 is not suitable for this application, as it is not a good representative of whether a student graduates after his program completion. This is simply because the feature at hand, i.e., the college applications are not relevant to the final prediction. A student's college application does not accurately suggest the student's performance over the course of his program. A better feature to serve as a representative of the population would be the grades of the Fall 2023 students, as they would provide a more accurate indication of whether the student graduates. Comparing the grades of Fall 2018 students to the grades of Fall 2023 students would make more sense, as the grade of a student is a far more reliable metric for predicting graduation outcome. Some other features for the classification could be passing grades for the various courses, difficulty of the various courses in the

program, etc, as these features would provide a better estimate of whether a student enrolled in this program graduates, than the college applications.

- C.** In this case, the training data is suitable for predicting whether the loan will be repaid. A financial application sheds light on the various aspects of a candidates' financial portfolio that suggest whether the candidate is fit to repay a loan. Labelled to whether the applicant repaid the loan, the financial applications of accepted candidates over the past 20 years will contain highly relevant features, such as Credit Score, Liquidity Ratio, etc, that suitably suggest why an accepted applicant was able to repay or not able to repay the loan. The relevance of the training data to the scenario at hand is very high, and therefore it is a strong representative of whether an accepted candidate will repay the loan.